Front-End Duo Guide — Procedure Drift Finder

For the two front-end builders · Streamlit (Python) · prepared 10 Oct 2026 · read the Team Playbook first for the big picture

1. Your mission

Build the screens a plant safety manager would use. Judges see the screens and the demo video, not the code, so clarity, evidence and polish win points. You build against **fake data first**, then switch to the real output from the back end on 15 Oct. You own features **F6, F7, F9 (panel), F13** and the container (**F10**) from the Playbook.

2. Who builds what

	Person A	Person B
Screens	Overview, Heat Strip (F7), Benchmark and Where we are wrong (F8), AMD and Evolus panel (F9, F12)	Review Queue (F6), Evidence Viewer (F13), SOP-fix box (F11), Demo mode
Shared	Visual style, Dockerfile (F10), README run steps, GitHub	

Rule: each person works on their own branch and merges once a day. Tell each other before changing shared files.

3. Setup checklist (finish before 12 Oct)

1. Install Python 3.11 or 3.12, VS Code and Git. Create a GitHub account.
2. In the project folder run `python -m venv .venv`, then activate it (Windows: `.venv\Scripts\activate`; Mac or Linux: `source .venv/bin/activate`).
3. Run `pip install streamlit pandas plotly`.
4. Start an app with `streamlit run app.py`.
5. Create a `.gitignore` containing `.venv`, `.env` and `.streamlit/secrets.toml`.

6. **AI helper:** Claude Code needs a paid plan or Console credits (the free chat plan does not include it). Free fallback: ask Claude chat for complete files, paste them into VS Code, run, and paste any error back. Official install, if you have access: macOS or Linux `curl -fsSL https://claude.ai/install.sh | bash`; Windows PowerShell `irm https://claude.ai/install.ps1 | iex`; then run `claude` inside the project folder. Start in Plan Mode so nothing changes until you approve.

4. Rules for working with an AI helper

- **Prompt formula:** who you are (beginner), the goal, the limits, how you will know it is done, and 'explain each file in one line'.
- One small step at a time. Run it after every step. Commit after every working step: `git add .`, `git commit -m 'what changed'`, `git push`.
- If something breaks, paste the **full error** and say 'fix only this'.
- Never paste or commit keys. Keys live in environment variables.
- Before relying on code you do not understand, send it to Claude chat and ask for a review.

5. Data format (agreed with the back end, do not change without asking)

- **flags:** flag_id, sop_id, step_id, note_id, date, shift, drift_type, quote, confidence (0 to 1), status (pending, approved or rejected), suggested_fix (text or empty).
- **drift_type values:** skipped_step, reordered_steps, workaround_substitution, sop_outdated_or_impractical, sop_unclear.
- **sop:** sop_id, step_id, title, text.
- **notes:** note_id, date, shift, source (handover, near-miss or maintenance), text.
- **benchmark:** precision, recall, quote_verified_rate, notes_per_minute, matcher_model, judge_model, gpu, false_alarms (list of flags the answer key says are wrong).
- Week numbers are calculated by the front end from the date.

6. The screens

1. **Overview:** counts (notes read, flags raised, verified, approved), the top 3 drifting steps, a banner saying Live or Demo mode, and the AMD panel summary.

2. **Heat Strip:** SOP steps down the side, weeks across, color shows the number of flags. Filters for SOP, shift and drift type. Choose a step to list its flags. Show a trend arrow.
3. **Review Queue:** filterable table. Selecting a flag opens a detail panel: SOP step text, note text with the quote highlighted, drift type, confidence, and Approve and Reject buttons. Decisions are saved.
4. **Evidence Viewer:** three columns: SOP step on the left, the highlighted quote in the middle, similar notes from other shifts on the right. This is the hero screen of the demo.
5. **SOP-fix box:** shown only for sop_outdated_or_impractical and sop_unclear flags. Label it clearly DRAFT, needs human approval.
6. **Benchmark:** four large numbers (precision, recall, quote-verified rate, notes per minute), a results table, and a Where our AI is wrong list.
7. **AMD and Evolus panel:** model names, GPU, endpoint status, throughput, cost per 1,000 notes, the Evolus building blocks used and the API call count.
8. **Demo mode toggle:** loads saved results when the live endpoint is off, and the banner always states which mode is active.

7. Design checklist (taste is judged)

- Three colors plus one warning color; one font; large numbers; generous spacing.
- The highlighted quote is the visual hero.
- Every page title answers one question, for example Which steps drift most?
- No personal names anywhere. Label the data as synthetic.
- Readable at video recording size (1080p).
- Every screen has an empty state, a loading state and an error message.

8. Day plan

Date	Person A	Person B
Oct 10–11	Setup, skeleton app with fake data, Overview	Setup, Review Queue on fake data
Oct 12	Sidebar filters, first commit, heat strip v1	Detail panel, quote highlight
Oct 13	Heat strip click-through, first Dockerfile attempt	Approve and reject saved

Date	Person A	Person B
Oct 14	Benchmark page	Evidence Viewer
Oct 15	AMD and Evolus panel, switch to real flags, clean-machine container test	SOP-fix box, Demo mode
Oct 16	Polish, error states, color check	Polish, full-flow test
Oct 17	Record the demo with marketing	Last bug fixes
Oct 18	Buffer	Buffer

9. Prompt pack (copy, paste, adjust)

Prompt 1: skeleton and fake data

> I am a complete beginner. Create a Streamlit app in Python with a sidebar menu for these pages: Overview, Heat Strip, Review Queue, Benchmark, AMD and Evolus. First create a data folder with realistic fake sop.json, notes.json, flags.json and benchmark.json (about 20 flags, using these fields: flag_id, sop_id, step_id, note_id, date, shift, drift_type, quote, confidence, status, suggested_fix). Use only streamlit, pandas and plotly. Each page can show its title and the raw data for now. Explain each file in one line and tell me how to run it.

Prompt 2: heat strip

> On the Heat Strip page show a plotly heatmap with SOP steps on the vertical axis and calendar weeks on the horizontal axis, colored by number of flags. Add sidebar filters for SOP, shift and drift type. Below the chart, let me pick a step from a dropdown and list its flags. Never show any person's name.

Prompt 3: review queue and evidence viewer

> On the Review Queue page show a filterable table of flags. When I pick one, show the SOP step text, the note text with the exact quote highlighted, the drift type, the confidence, and Approve and Reject buttons that update the status and save to data/decisions.json. Add an Evidence Viewer with three columns: SOP step, highlighted quote, and similar notes from other shifts. For sop_outdated_or_impractical and sop_unclear flags, show the suggested fix in a box labelled DRAFT, needs human approval.

Prompt 4: benchmark

> On the Benchmark page show precision, recall, quote-verified rate and notes per minute as large numbers from benchmark.json, then a table titled Where our AI is wrong listing the false alarms.

Prompt 5: demo mode

> Add a Demo mode toggle that loads saved results from the data folder when the live AI endpoint is not reachable. Show a banner on every page saying which mode is active.

Prompt 6: container

> Write a Dockerfile and requirements.txt for this Streamlit app. It must run on linux/amd64, listen on 0.0.0.0 port 8501 and read secrets from environment variables. Show me the exact commands to build it (using docker build with the platform option for linux/amd64) and run it, and how to check it works.

10. Container test (feature F10)

- Install Docker Desktop. Build and run with the commands from Prompt 6. Open http://localhost:8501.
- Test on a different laptop if you can. Keep keys out of the image and the repo.
- Tell Builder A the image name. In the previous AMD hackathon, 248 submissions failed automated scoring, mostly from container problems, so do not leave this to the last day.

11. Definition of done (before recording the demo)

- Every screen works in both Live and Demo mode.
- The quote highlight and the heat strip click-through work without errors.
- No names, keys or endpoint addresses visible anywhere.
- The container builds and starts on a clean machine.
- Benchmark numbers on screen match benchmark.json from the real run.

12. What to bring to Claude for review

Paste any file you do not understand, any error message, and screenshots of each screen. Ask: review this for bugs, leaked keys, and anything that would break inside Docker.